

Barcode-Kamerasystem

Systemdokumentation — 8-Bahn-Inspektionsanlage

1. Aufgabenstellung

Automatische Erfassung und Verifikation von Interleaved-2/5-Barcodes auf bis zu 8 parallel laufenden Bahnen. Pro Bahn werden 4 Barcodes ($24,2 \times 10,5$ mm) gelesen und mit dem aufgedruckten Klartext verglichen. Die Ergebnisse werden zentral protokolliert. Die Bahnen laufen synchron mit 49 m/min.

Parameter	Wert
Bahngeschwindigkeit	49 m/min (816 mm/s)
Anzahl Bahnen	6 oder 8
Barcodes pro Bahn	4 × Interleaved 2/5
Barcode-Größe	24,2 × 10,5 mm
Barcode-Orientierung	Striche quer zur Laufrichtung
Klartext-Vergleich	OCR aus demselben Kamerabild
Sequenzprüfung	fortlaufende Nummerierung
Fehlerreaktion	Protokollierung (kein Bahnstop)

2. Gesamtarchitektur

Das System besteht aus 4 identischen Kamera-Einheiten (je 2 Bahnen) und einem zentralen Protokoll-Server. Alle Einheiten werden von einem gemeinsamen Trigger ausgelöst und sind über einen LAN-Switch verbunden.

Systemstruktur:

```
OPB917BZ (gemeinsamer Trigger)
■
■■■ CM5 #1 (Bahn 1 + 2) ■■■
■■■ CM5 #2 (Bahn 3 + 4) ■■■
■■■ CM5 #3 (Bahn 5 + 6) ■■■■ LAN-Switch ■■ Pi5 #5 (Server + SSD)
■■■ CM5 #4 (Bahn 7 + 8) ■■■
```

Hardwareübersicht:

Einheit	Hardware	Aufgabe
CM5 #1–4	Raspberry Pi CM5 + Hailo-8L (13 TOPS) + 2× IMX296 Global Shutter	Bilderfassung, Barcode-dekodierung, OCR, Vergleich
Pi5 #5	Raspberry Pi 5 + SSD (USB3)	MQTT-Broker, Protokoll-server, Datenbankschreiber
Trigger	OPB917BZ + BC849 + 560 Ω	Bahnkantenerkennung, GPIO-Signal an alle CM5
Beleuchtung	LED-Blitz (Eigenbau) IRLZ44N + High-Power LED	Kurzzeitbeleuchtung 50–100 µs Pulsdauer
Netzwerk	Gigabit-Switch 5 Ports	Verbindung aller Einheiten

3. Kamera-Einheit (CM5 #1–4)

Jede CM5-Einheit ist identisch aufgebaut und verarbeitet 2 Bahnen unabhängig.

Aufbau je Einheit:

```
CM5-Einheit
■■■ CAM0 → Global Shutter Kamera (Bahn A, 4 Barcodes)
■■■ CAM1 → Global Shutter Kamera (Bahn B, 4 Barcodes)
■■■ GPIO (Trigger-Eingang) ← OPB917
■■■ GPIO (LED-Blitz-Ausgang) → IRLZ44N
■■■ Hailo-8L HAT+ (OCR-Inferenz)
■■■ LAN → Switch → Pi5 #5
```

Kamera-Spezifikation:

Parameter	Wert
Sensor	Sony IMX296 (Global Shutter)
Auflösung	1456 × 1088 Pixel
Pixelgröße	3,45 µm
Max. Framerate	60 fps
Schnittstelle	CSI-2 (MIPI, 2-Lane)
Auflösung quer (2 Barcodes)	43,5 px/mm → 17 Pixel/Modul
Auflösung längs (Barcode)	60 px/mm → 24 Pixel/Modul

4. Trigger-Konzept

Ein gemeinsamer OPB917BZ-Sensor erkennt die Bahnvorderkante. Das Signal wird als Stern an alle 4 CM5-Einheiten verteilt. Jede CM5 berechnet die Verzögerung bis das Etikett im Kamerafeld ist und löst dann Blitz und Kameras aus.

Signalfluss:

```
OPB917BZ Ausgang (Open-Collector)
■
BC849 (560 Ω Basis) → Pull-up 10 kΩ / 5V
■
Sternverteiler ■■■ GPIO CM5 #1 (Interrupt)
■■■ GPIO CM5 #2 (Interrupt)
■■■ GPIO CM5 #3 (Interrupt)
■■■ GPIO CM5 #4 (Interrupt)
■
Verzögerung (Offset-Zeit berechnen)
■
GPIO-Puls → LED-Blitz + Kamera-Trigger gleichzeitig
```

OPB917BZ Anschlusswerte:

Bauteil	Wert	Bemerkung
LED-Vorwiderstand	560 Ω / 5V	I _F = 5,9 mA, sicher im Schaltbereich
Phototransistor Pull-up	10 kΩ / 5V	Ausgangspegel 0–5V
BC849 Basis-Widerstand	100 kΩ	Kollektorstrom ~66 µA (GP Pull-up)
VCC Sensor-IC	5V (VSYS)	Min. 4,5V laut Datenblatt
GP6 Konfiguration	Input + Pull-up intern	RP2350 ~50 kΩ intern

5. LED-Blitz (Eigenbau)

Pro CM5-Einheit ein LED-Blitz der beide Bahnen gleichzeitig beleuchtet. Der Pi5 GPIO steuert direkt einen Logic-Level MOSFET.

Schaltungsprinzip:

```
Pi5 GPIO (3,3V)
■
10 Ω (Gate-Dämpfung)
■
Gate ■■ IRLZ44N (Logic-Level MOSFET)
Source → GND
Drain → LED-Array (Kathode)
■
Vorwiderstand (Strombegrenzung)
■
1000 µF Pufferkondensator
■
12V / 24V Versorgung
```

Bauteilliste je Blitz-Einheit:

Bauteil	Wert / Typ	Funktion
MOSFET	IRLZ44N	Logic-Level, direkt von 3,3V steuerbar, 55V/47A
Gate-Widerstand	10 Ω	HF-Schwingungen am Gate unterdrücken
Pufferkondensator	1000 µF / 25V	Pulsstrom liefern ohne Spannungseinbruch
LED-Array	High-Power weiß 3W	z.B. 3–4× in Reihe je nach Versorgung
Vorwiderstand	je nach Pulsstrom	$R = (U_{VCC} - U_{LED}) / I_{Puls}$
Freilaufdiode	1N4007	Schutz bei induktiven Anteilen

Überstrombetrieb im Puls:

Bei 100 µs Pulsdauer und 100 Hz Wiederholrate beträgt das Tastverhältnis nur 1 %. LEDs dürfen im Pulsbetrieb kurzzeitig mit ca. 10× Nennstrom betrieben werden — die mittlere thermische Belastung bleibt im sicheren Bereich.

```
Beispiel: 1W LED (I_N = 350 mA) → Pulsstrom bis 3,5 A zulässig
Vorwiderstand (12V, 3× LED in Reihe, I = 1A): R = (12 - 9,6) / 1 = 2,4 Ω → 2,2 Ω
```

Maximale Bahngeschwindigkeit:

LED-Blitz Pulsdauer	Max. Bahngeschwindigkeit	Engpass?
100 µs	60 m/min	Ja — Hauptlimiter
50 µs	120 m/min	Ja — Hauptlimiter
20 µs	300 m/min	Ja — Hauptlimiter
Kamera 60 fps	~1800 m/min*	Nein
Hailo + Pi5 CPU	~300 m/min*	Nein

* bei 500 mm Etikettenabstand

6. Software-Ablauf je CM5-Einheit

Verarbeitungsablauf pro Auslösung:

1. GPIO-Interrupt (OPB917 Trigger)
2. Verzögerung berechnen (Offset bis Kamerafeld)
3. LED-Blitz auslösen + beide Kameras triggern
4. Bild 1 (Bahn A) und Bild 2 (Bahn B) erfassen
5. Je Bild: 4 Spalten aufteilen (je 1 Barcode)
6. Pro Spalte:
 - a) Oberer Bereich → Hailo OCR (Klartext)
 - b) Unterer Bereich → pyzbar (ITF Barcode)
 - c) Vergleich: OCR == Barcode?
7. Sequenzprüfung: fortlaufende Nummerierung?
8. Ergebnis per MQTT an Pi5 #5 senden

Software-Stack je CM5:

Komponente	Software	Aufgabe
Bilderfassung	libcamera / Picamera2	Beide CSI-Kameras gleichzeitig
Barcode-Lesen	pyzbar (Python)	ITF Interleaved 2/5 dekodieren
OCR	CRNN / PaddleOCR auf Hailo	Klartext aus Bild, nur Ziffern
Kommunikation	paho-mqtt (Python)	Ergebnisse an MQTT-Broker senden
Trigger	RPi.GPIO / gpiod	GPIO-Interrupt, Puls-Ausgabe

7. Zentraler Protokoll-Server (Pi5 #5)

Der Pi5 #5 läuft als MQTT-Broker und Datenbankserver. Alle CM5-Einheiten verbinden sich beim Start und publizieren ihre Ergebnisse asynchron.

MQTT-Themen-Struktur:

```
scanner/einheit1/bahn1/ergebnis
scanner/einheit1/bahn2/ergebnis
scanner/einheit2/bahn1/ergebnis
...
scanner/einheit4/bahn2/ergebnis
```

Protokoll-Datensatz (JSON je Auslösung):

```
{
  "timestamp": "2026-07-02T08:15:32.412",
  "einheit": 1,
  "bahn": 1,
  "barcodes": [
    {"nr": 1, "barcode": "5201881001.07", "ocr": "5201881001.07", "ok": true},
    {"nr": 2, "barcode": "5201881001.08", "ocr": "5201881001.08", "ok": true},
    {"nr": 3, "barcode": "5201881001.09", "ocr": "5201881001.09", "ok": true},
    {"nr": 4, "barcode": "5201881001.10", "ocr": "5201881001.10", "ok": true}
  ],
  "sequenz_ok": true,
  "gesamt_ok": true
}
```

Server-Software:

Komponente	Software	Aufgabe
------------	----------	---------

MQTT-Broker	Mosquitto	Nachrichten empfangen und verteilen
Datenbank	SQLite auf SSD	Alle Ergebnisse persistent speichern
Protokoll-Schreiber	Python + paho-mqtt	MQTT subscriben → SQLite schreiben
Webinterface	Flask (optional)	Protokoll im Browser anzeigen

8. Stückliste Gesamtsystem

Pos.	Bauteil	Menge	Bemerkung
1	Raspberry Pi CM5	4	je eine Kamera-Einheit
2	Hailo-8L AI HAT+	4	13 TOPS, je CM5
3	RPi Global Shutter Camera IMX296	8	2 je CM5-Einheit
4	Raspberry Pi 5	1	Protokoll-Server
5	SSD (USB3, mind. 256 GB)	1	Protokoll-Speicher
6	Gigabit-Switch (5-Port)	1	LAN-Verbindung alle Einheiten
7	OPB917BZ	1	Gemeinsamer Trigger
8	BC849 NPN-Transistor	1	Trigger-Signalaufbereitung
9	IRLZ44N MOSFET	4	1 je LED-Blitz-Einheit
10	High-Power LED 3W weiß	16	4 je Blitz-Einheit
11	Elko 1000 µF / 25V	4	Pufferkondensator je Blitz
12	Widerstände (diverse)	nach Schaltplan	